

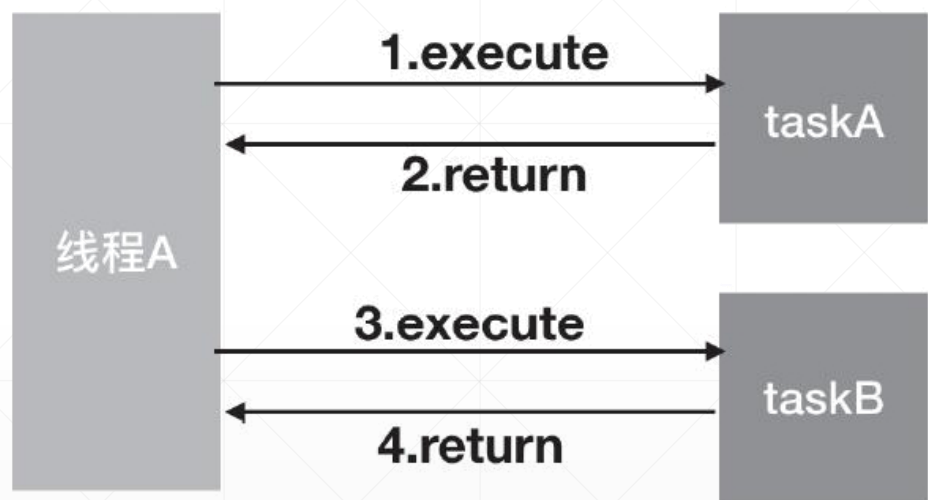
CompletableFuture之一



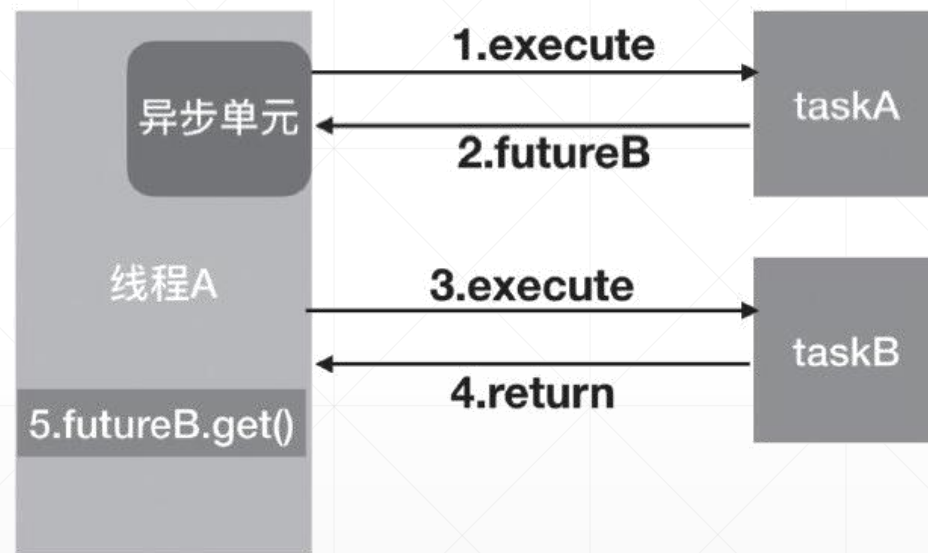
从“同步”到“异步”

北京理工大学计算机学院
金旭亮

同步与异步



同步调用



异步调用



异步单元的执行一般是由线程池中的线程执行。

通过代码理解同步与异步-1

//用于模拟一个计算任务

4 usages

```
static void task(String taskId) {  
    System.out.println(taskId);  
    var threadName :String = Thread.currentThread().getName();  
    for (int i = 1; i <= 10; i++) {  
        System.out.println(threadName + "线程报告:" + taskId + "已完成" + (i * 10) + "%");  
        try {  
            Thread.sleep( millis: 500);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

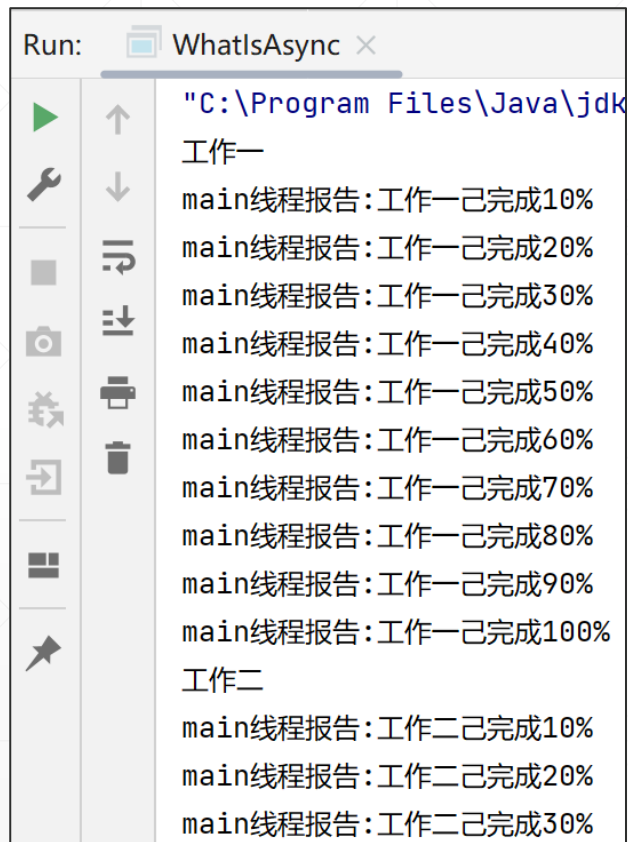
通过代码理解同步与异步-2

//同步执行的代码

1 usage

```
private static void syncCode() {  
    task( taskId: "工作一");  
    task( taskId: "工作二");  
}
```

代码按顺序执行，并且是由同一个线程负责执行.....



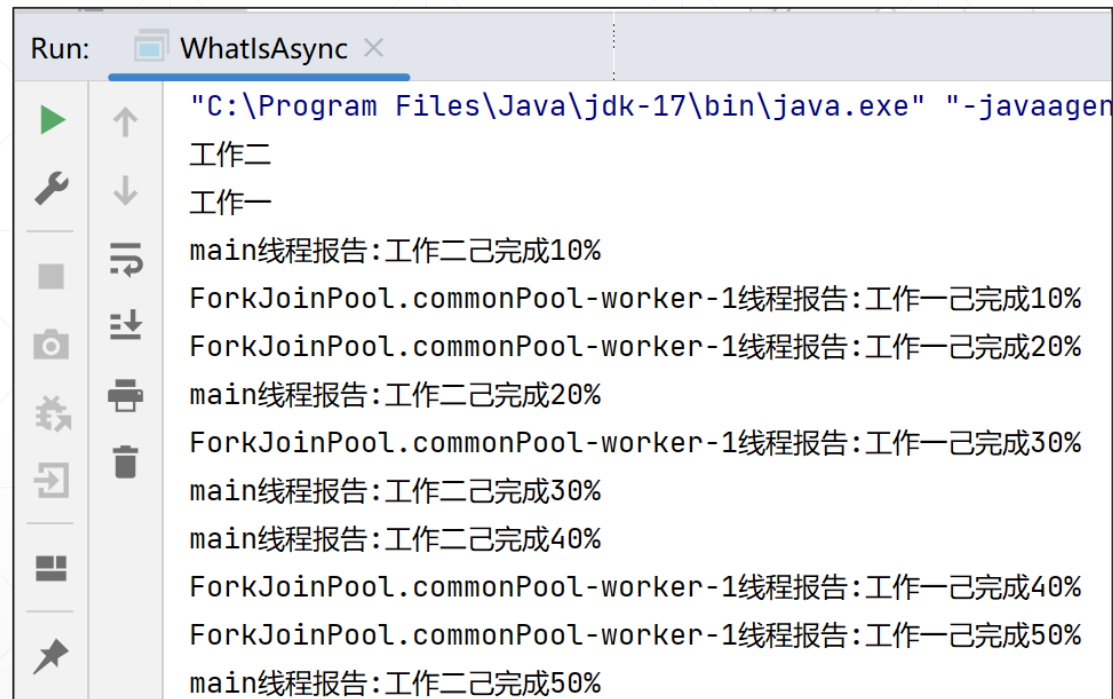
通过代码理解同步与异步-3

//异步执行的代码

1 usage

```
private static void asyncCode() {  
    CompletableFuture.runAsync(() -> {  
        task( taskId: "工作一");  
    });  
    task( taskId: "工作二");  
}
```

异步执行的代码，两个计算任务是“交替”且“乱序”执行的，并且由两个不同的线程执行.....



Run: WhatIsAsync x

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagen  
工作二  
工作一  
main线程报告:工作二已完成10%  
ForkJoinPool.commonPool-worker-1线程报告:工作一已完成10%  
ForkJoinPool.commonPool-worker-1线程报告:工作一已完成20%  
main线程报告:工作二已完成20%  
ForkJoinPool.commonPool-worker-1线程报告:工作一已完成30%  
main线程报告:工作二已完成30%  
main线程报告:工作二已完成40%  
ForkJoinPool.commonPool-worker-1线程报告:工作一已完成40%  
ForkJoinPool.commonPool-worker-1线程报告:工作一已完成50%  
main线程报告:工作二已完成50%
```

概述



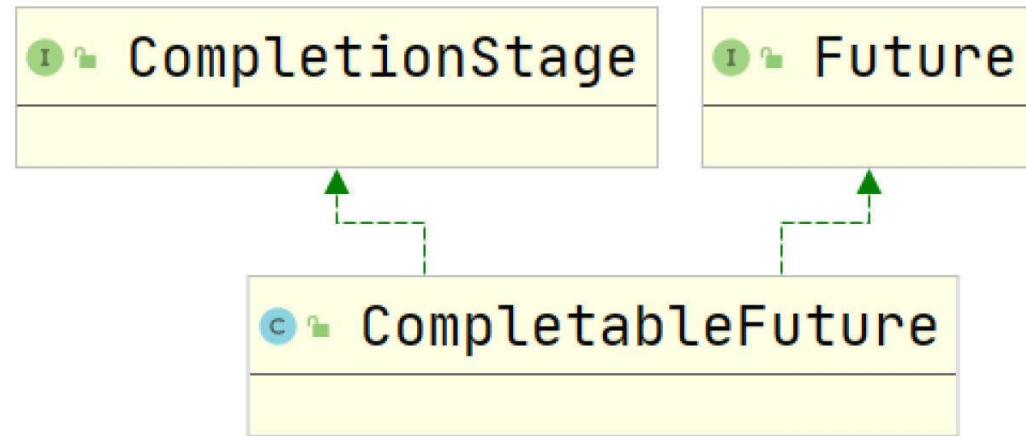
在Java中，可以使用Future在某个未来时刻获取异步任务的执行结果。但是获取其结果时还是会阻塞调用线程的，并没有实现完全的异步化处理，所以在JDK8中提供了CompletableFuture来弥补其缺点。



CompletableFuture类允许以非阻塞的基于通知的方式处理结果，通过设置一个回调函数，让主线程彻底解放出来，实现了真正意义上的异步处理。



CompletableFuture的常用场景之一就是异步执行一段代码，该段代码计算并返回一个值，典型例子就是将它用于实现异步IO操作。



`CompletionStage`表示任务执行的一个阶段，这个接口定义非常多的方法，如`thenApply()`、`thenAccept()`等，通过这些方法可以实现多个任务之间的时序关系，如串行、并行、聚合。

学用CompletableFuture



CompletableFuture向开发者提供了一个异步的、响应式的编程模型。

CompletableFuture是一个可以通过编程方式显式地设置计算结果和状态以便让任务结束的特殊的Future对象，它的功能比较强大，比如，计算完成时可以触发一个函数或者行为；另外，它是线程安全的。

基于CompletableFuture的异步编程模型



可以将每个CompletableFuture对象看成一个异步操作。



多个异步操作之间可以有三种关联方式：

(1) forked: 分支

(2) chained: 级联

(3) joined: 等待完成



默认情况下，CompletableFuture对象使用ForkJoin线程池，但也可以指定它使用其他类型的线程池。

“级联调用”的异步操作示例

```
CompletableFuture.supplyAsync(...)  
    .thenApply(...)  
    .thenAccept(...)
```

上述代码中的每一步操作（称为“completion stage”）都是异步的，不会阻塞调用者线程，并且后一步在前一步完成后自动执行。



使用Completable Future框架开发，基本上就不需要理会Thread、线程同步和锁等那些复杂的技术细节。

下面我们通过示例展示CompletableFuture的基本用法.....

场景一：异步运行不返回值的后台任务

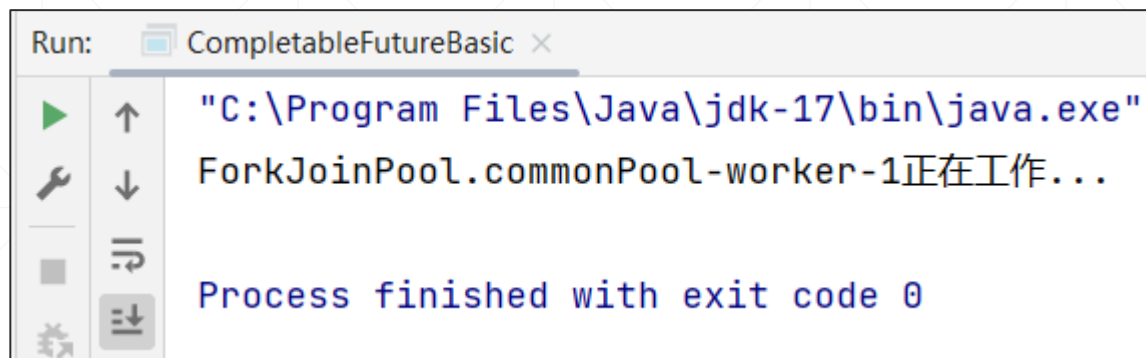
使用Completable的**runAsync()**方法，传给它一个Lambda表达式，就能将工作任务提交给线程池中的线程，在后台执行，注意这种工作任务，是没有返回值的.....

```
private static void runAsync() {  
    //以异步方式运行一个任务  
    var future : CompletableFuture<Void> = CompletableFuture.runAsync(() -> {  
        var threadName : String = Thread.currentThread().getName();  
        System.out.println(threadName + "正在工作...");  
    });  
    //等待任务的完成  
    future.join();  
}
```

CompletableFutureBasic.java

调用**join()**方法可以等待后台任务执行完毕。

示例输出



The screenshot shows a 'Run' window with a tab titled 'CompletableFutureBasic'. The console output is as follows:

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
ForkJoinPool.commonPool-worker-1正在工作...  
  
Process finished with exit code 0
```

从程序输出中可以看到，CompletableFuture底层使用的是Fork-Join线程池。

如果需要使用其他类型的线程池.....



默认情况下支撑CompletableFuture异步运行的是JDK中的一种比较特殊的线程池——ForkJoinPool，但如果需要，也可以设置为使用其他的线程池。



使用runAsync方法的以下重载形式，可以指定用线程池来运行异步任务：

```
public static CompletableFuture<Void> runAsync(Runnable runnable,  
                                              Executor executor)
```



JDK中内置了一些典型的线程池，将在另一讲PPT中介绍。

更换使用的线程池

```
private static void runAsyncUseOtherThreadPool() {  
    var threadPool : ExecutorService = Executors.newCachedThreadPool();  
    //以异步方式运行一个任务  
    var future : CompletableFuture<Void> = CompletableFuture.runAsync(() -> {  
        var threadName : String = Thread.currentThread().getName();  
        //输出: pool-1-thread-1正在工作...  
        System.out.println(threadName + "正在工作...");  
    }, threadPool);  
    //等待任务的完成  
    future.join();  
}
```

场景二：异步运行有返回值的后台任务



将返回值的任务封装为一个Supplier对象，然后，将它传给 **supplyAsync()** 方法，就能在后台执行这个任务，这种任务可以有返回值：

```
private static void supplyAsync()  
    throws InterruptedException, ExecutionException {  
    Supplier<Integer> task = () -> new Random().nextInt();  
    var future : CompletableFuture<Integer> = CompletableFuture.supplyAsync(task);  
    var result : Integer = future.get();  
    System.out.println(result);  
}
```



需要取回结果时，只需要调用 **get()** 方法，调用者线程将在此阻塞等待后台任务完成，然后取回结果。

异步方法的命名规范

通常而言，名称中不带Async的方法和它的前一个任务一样，在同一个线程中运行，而名称以Async结尾的方法会将后续任务提交到一个线程池，所以每个任务是由不同的线程处理的。

场景三：将同步代码转换为异步代码

下面，通过一个需要执行较长时间的同步任务，展示如何利用 `CompletableFuture` 让它以异步方式运行：

```
//模拟一个需要执行较长时间的同步任务
```

```
private static void longTask() throws InterruptedException {  
    var currentThread :String =Thread.currentThread().getName();  
    System.out.println(currentThread+"线程正在干活，需要耗费一定的时间");  
    Thread.sleep( millis: 3000);  
    System.out.println(currentThread+"线程的工作已经完成。");  
}
```

SyncToAsync.java

同步方法，异步执行.....

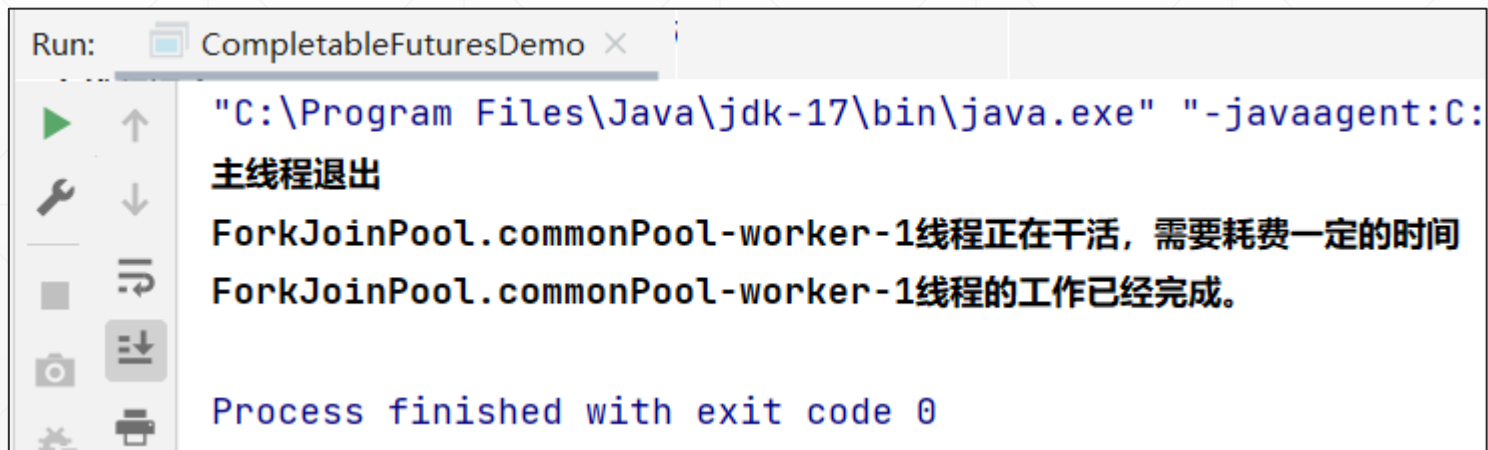
//封装同步方法为异步调用

```
private static CompletableFuture<Void> syncToAsync(){  
    return CompletableFuture.runAsync(()-> {  
        try {  
            longTask(); //同步方法，封装了要执行的工作任务  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    });  
}
```

如果同步方法是有返回值的，可以使用supplyAsync()进行封装。

```
public static void main(String[] args)  
    throws ExecutionException, InterruptedException {  
    var asyncTask = syncToAsync();  
    System.out.println("主线程退出");  
    asyncTask.join();  
}
```

运行截图：



```
Run: CompletableFuturesDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:  
主线程退出  
ForkJoinPool.commonPool-worker-1线程正在干活，需要耗费一定的时间  
ForkJoinPool.commonPool-worker-1线程的工作已经完成。  
Process finished with exit code 0
```

场景四：改造传统多线程代码为异步代码



使用`runAsync()`或`supplyAsync()`可以很方便地将同步代码封装为异步代码。



但对于那些本身已经是多线程运行的代码，你可能想将其改造为使用`CompletableFuture`的异步代码，这时，可以在线程外部创建一个`CompletableFuture`实例，然后，显式设置它的结果，从而，让外界可以以一致的异步调用方式执行这些传统的多线程代码。

"多线程转异步"示例-1

ExplicitSetValue.java

```
public class ExplicitSetValue {  
    public static void main(String[] args) throws ExecutionException,  
        InterruptedException {  
        //创建线程池  
        var threadPool : ExecutorService = Executors.newFixedThreadPool( nThreads: 2);  
        //创建一个CompletableFuture对象, 使用它来承载计算结果  
        CompletableFuture<String> future = new CompletableFuture<>();  
        //向线程池提交计算任务, 并设置结果  
        threadPool.execute(() -> {  
            longTask(future);  
        });  
        System.out.println("---主线程等待计算结果---");  
        //get方法会阻塞主线程, 直到计算结果, 得到了结果  
        var result : String = future.get();  
        System.out.println("---主线程得到了结果---");  
        System.out.println(result);  
        //关闭线程池  
        threadPool.shutdown();  
    }  
    1 usage  
    private static void longTask(CompletableFuture<String> future) {...}  
}
```

Run: ExplicitSetValue x

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
---主线程等待计算结果---  
线程池中的pool-1-thread-1正在处理数据  
----pool-1-thread-1得到了最终处理结果 ----  
---主线程得到了结果---  
这是最终结果: 679969203  
  
Process finished with exit code 0
```

“多线程转异步”示例-2

```
private static void longTask(CompletableFuture<String> future) {  
    //休眠3s, 模拟任务计算  
    try {  
        var threadName : String = Thread.currentThread().getName();  
        System.out.println("线程池中的"+threadName+"正在处理数据");  
        Thread.sleep( millis: 3000);  
        //生成一个随机数作为结果  
        int result = new Random(System.currentTimeMillis()).nextInt();  
        System.out.println("----" +threadName  + "得到了最终处理结果 ----");  
        //显式设置结果  
        future.complete( value: "这是最终结果: " + result);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

通过调用complete()方法显式设置CompletableFuture<T>的值。

ExplicitSetValue.java

理解complete()方法的作用

当你已经有了一个CompletableFuture对象，调用它的get()或join()方法，总是阻塞的.....

```
future.join();    //调用者线程在这里阻塞等待
```

如果你在另外一个线程中调用了CompletableFuture对象complete()方法，则上述join()方法将立即返回，从而让调用者线程可以继续执行。